2024 年度卒業 修士論文

Learning-Augmented Caching: From Theoretical Guarantees to Practical Performance

23M30425 **色部 浩男**

指導教員　金森 敬文 教授

January, 2025

東京科学大学　情報理工学院
数理・計算科学系

Abstract

This thesis explores the design of online caching algorithms augmented with machine learning (ML) predictions. Recent years have seen growing interest in such learning-augmented algorithms which aim to outperform classical bounds by leveraging ML predictions while maintaining robust to poor predictions. For the online paging problem, existing approaches typically achieve these guarantees by dynamically switching between classical eviction rules and prediction-based decisions. We identify fundamental limitations in this paradigm - although it ensures robustness, it fails to realize the full potential of integrating classical algorithms with learned predictions.

This thesis proposes a simpler and more practical approach that combines predictions with classical rules more seamlessly. Our key insight is that these signals provide complementary information that should be combined rather than alternated between. We provide theoretical analyses showing that this weighted integration can reduce variance in the predicted distance estimates, leading to more effective eviction decisions while maintaining strong worst-case bounds similar to those in prior literature. Initial experiments on standard caching benchmarks validate this insight, demonstrating that appropriate weighted combinations significantly outperform both pure strategies across diverse workloads.

A key challenge in deploying such weighted integration is determining appropriate weights for combining the signals in an online setting. The optimal weight varies across workloads and may change dynamically as access patterns evolve. We develop and analyze two approaches: an efficient golden-section search leveraging empirically observed unimodality, and a more robust Gaussian process-based method that handles noisy observations and changing distributions. Further experimental evaluation shows our online parameter tuning methods often achieve the performance of offline optimal weight while maintaining computational efficiency.

This work suggests broader implications for algorithm design, showing how thoughtfully integrating ML predictions with classical algorithmic principles can achieve better performance than treating them as competing alternatives. We believe similar principles could improve learning-augmented algorithms for other fundamental problems beyond caching.

Contents

Notations

Caching Related	
k	Cache size
$\sigma = (\sigma_1, \sigma_2, ..., \sigma_n)$	Request sequence of length n
S	Current cache contents
$l(x)$	Last access time of page x
$n(x)$	Next requested time of page x
$d(x)$	True reuse distance for page x
$h(x)$	ML-predicted reuse distance for page x
OPT	Optimal offline cost
$\text{cost}_{\mathcal{A}}$	Cost of algorithm $\mathcal{A}$
$\text{CR}_{\mathcal{A}}$	Competitive ratio of algorithm $\mathcal{A}$
Statistical Notation	
$\mathbb{E}[\cdot]$	Expected value
$\mathbb{P}(\cdot)$	Probability
$\mathbb{V}[\cdot]$	Variance
$\Phi(\cdot)$	Standard normal CDF
$\phi(\cdot)$	Standard normal PDF
Miscellaneous Notations	
η	Prediction error
α	Weighting parameter between ML and LRU
W	Window size for parameter tuning
M	Update period for parameter tuning
K	Sliding window size for GP observations

Chapter 1

Introduction

The classical paradigm of algorithm design has focused predominantly on developing algorithms that provide worst-case performance guarantees. Although this approach has led to many fundamental advances, it often results in overly conservative performance bounds that fail to capture the empirical behavior of algorithms on real-world instances. This disconnect has motivated the broader movement of "beyond worst-case analysis"[70] that seeks to develop more nuanced theoretical frameworks for algorithm analysis. While various approaches have been proposed - from smoothed analysis to instance-specific bounds - a particularly promising direction has emerged in recent years: "learning-augmented algorithms"[62]. This paradigm augments traditional algorithms with machine-learned predictions to achieve better performance on typical instances while maintaining worst-case robustness guarantees.

The online paging problem (also known as caching)[79] represents an ideal testing ground for learning-augmented algorithms. In this fundamental problem, we must manage a cache of size k to serve a sequence of page requests, evicting pages when the cache is full. It is a classical result that no deterministic online algorithm can achieve a competitive ratio better than k, while the randomized lower bound is H_k (the k-th harmonic number)[79, 39]. These bounds reflect the inherent difficulty of making eviction decisions without knowledge of future requests.

Practical applications, however, generate page request sequences that exhibit rich structure - from simple temporal and spatial locality to complex, application-specific memory access patterns. Recent work[60, 69, 84, 32] has explored learning-augmented approaches for caching that utilize predictions of reuse distances - when pages will be requested again in the future. Given perfect predictions, these algorithms can match Bélády's optimal offline policy[25] of evicting the page needed furthest in the future. To handle imperfect predictions, following the framework of Mitzenmacher and Vassilvitskii (2022)[62], existing approaches ensure robustness (behaving comparably to the best online algorithm given any predictions) and consistency (approaching optimal performance given perfect predictions) at the same time. These guarantees are essentially achieved by dynamically switching between the following two modes:

(i) Following classical eviction rules (e.g., LRU, MARKER, RANDOM)
(ii) Trusting ML predictions completely (known as BlindOracle)

, based on recent or historically observed performance. While this ensures robustness against worst-case inputs, it fails to realize the full potential of integrating classical algorithms with learned predictions. As demonstrated in comprehensive experiments by Chłędowski et al.[32], most algorithms struggle to consistently outperform the better of their two components - the classical algorithm and the ML predictor. We identify that this limitation arises from treating these components as competing alternatives rather than complementary signals: Classical heuristics like LRU capture fundamental properties like temporal locality, while ML predictions can identify complex, application-specific patterns.

This thesis proposes a fundamentally different approach to learning-augmented caching that maintains theoretical guarantees while significantly improving empirical performance. Our key insight is that classical eviction policies and learned predictions provide complementary signals that should be combined rather than alternated between. We formalize this through weighted combination, introducing a parameter α that determines the relative weight given to LRU compared to ML predictions. Figure 1.1 shows experimental results on standard caching benchmarks that validate this insight - plotting cache hit rate against our weighting parameter α reveals a clear peak at intermediate values, showing that true integration outperforms either signal alone. This finding stands in contrast to previous approaches[60, 69, 84] that either switch discretely between strategies and/or, in the case of black-box combiners[40, 75], eventually follow their better-performing component exclusively[84]. By properly choosing α, our approach could consistently outperform both the classical algorithm and the pure ML predictor alone.

While our findings show promise, it introduces a new challenge: good α is unknown in advance, how should we select the appropriate weighting parameter α in an online setting? We also need to put the following into consideration:

(i) The optimal α varies significantly across different workloads and may change dynamically within a single workload as access patterns evolve
(ii) While the performance curve exhibits clean, approximately unimodal structure when evaluated on complete sequences, limited data during online learning leads to high-variance estimates.
(iii) The computational overhead of parameter tuning must remain minimal for practical deployment.

This challenge connects to the broader field of data-driven algorithm design[46, 10], where algorithm parameters are tuned using problem instances to achieve good expected performance. While theoretical frameworks exist specifically for online parameter tuning[77], the direct application of these methods to our setting is impractical due to the computational intractability of their algorithms. These methods assume adversarial environment shifts and general function classes, whereas our setting exhibits more structure - approximate unimodality and gradual rather than arbitrary environment changes - suggesting more efficient solutions are possible.

This paper's contributions are summarized as follows:

(i) We identify fundamental limitations in existing learning-augmented caching al-

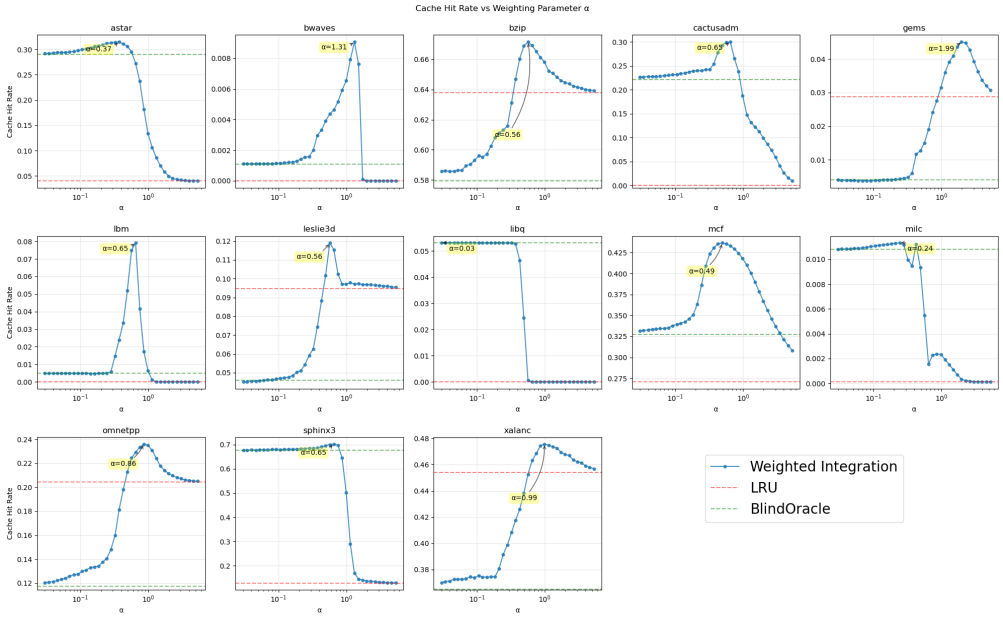

Fig. 1.1: Empirical evaluation of the weighted integration on SPEC CPU2006 benchmarks [47]. Combining the two signals with appropriate weighting outperforms both alone. The performance curves exhibit approximate unimodality.

gorithms and propose a novel approach to significantly improves empirical performance while maintaining worst-case guarantees through the black-box combiner framework in [84]. We demonstrate its effectiveness both empirically and theoretically.

(ii) We develop and analyze practical online parameter tuning methods based on Gaussian processes that achieve near-optimal performance with reasonable computational overhead.

Chapter 2

Preliminaries

2.0.1 Classical Online Caching

The online paging problem, also known as caching, represents one of the most fundamental and well-studied problems in online algorithms. Consider a two-level memory hierarchy consisting of a fast memory (cache) of size k and a slower but larger main memory. A sequence of page requests arrives online, where each request must be served from the cache. If the requested page is not present in the cache (a cache miss), it must be brought into the cache, potentially evicting another page to make room. The goal is to minimize the number of cache misses over the request sequence. Formally, let $\sigma = (\sigma_1, \sigma_2, ..., \sigma_n)$ denote a sequence of page requests. At each time t, an online algorithm $\mathcal{A}$ must decide which page to evict based only on the requests $\sigma_1, ..., \sigma_t$ seen so far. Let $\text{cost}_{\mathcal{A}}(\sigma)$ denote the number of cache misses incurred by algorithm $\mathcal{A}$ on sequence σ, and let $\text{OPT}(\sigma)$ denote the minimum possible number of cache misses achievable by any algorithm with full knowledge of σ. The performance of online algorithms is typically measured through competitive analysis:

Definition 2.0.1 (Competitive Ratio)**.** An online algorithm $\mathcal{A}$ is c-competitive if for all request sequences σ:

$$\text{cost}_{\mathcal{A}}(\sigma) \leq c \cdot \text{OPT}(\sigma) + \alpha$$

where α is a constant independent of the input length. The competitive ratio of $\mathcal{A}$ is the infimum over all such c, which we denote by $\text{CR}_{\mathcal{A}}$.

A classical result by Sleator and Tarjan (1985)[79] shows that no deterministic online algorithm can achieve a competitive ratio better than k for a cache of size k. Two classical algorithms are shown below:

Least Recently Used (LRU) A widely deployed algorithm which evicts the page that was least recently requested. It achieves the optimal competitive ratio of k.

First In First Out (FIFO) This algorithm evicts pages in the order they entered the cache. FIFO also achieves a competitive ratio of k.

Another important result is the randomized MARKER algorithm introduced by Fiat et al. (1991)[39]. For each page in cache, MARKER maintains a single bit indicating whether the page is "marked". All pages start unmarked at the beginning of a

phase. When a page is requested, it becomes marked. Upon a cache miss with no unmarked pages remaining, all marks are cleared, starting a new phase. On a cache miss, MARKER evicts a uniformly random unmarked page. This simple algorithm achieves:

Theorem 2.0.1 ([39])**.** The randomized MARKER algorithm achieves a competitive ratio of $2H_k$ in expectation, where H_k is the k-th harmonic number.

This bound is optimal up to constant factors, as Fiat et al. [39] also proved that no randomized algorithm can achieve a competitive ratio better than H_k.

For the offline version of the problem, Belady (1966)[25] proved that the optimal strategy is to evict the page that will be requested furthest in the future:

Theorem 2.0.2 ([25])**.** For any request sequence σ, the algorithm that always evicts the page whose next request appears furthest in the future achieves the minimum possible number of cache misses.

2.0.2 Learning-Augmented Algorithms

The framework of learning-augmented algorithms, introduced by Mitzenmacher and Vassilvitskii [62], represents a fundamentally new approach to algorithm design that leverages machine learning predictions while maintaining worst-case guarantees. To illustrate the key ideas, we begin with a simple example. Consider the classical binary search problem on a sorted array A of length n. The standard algorithm achieves $O(\log n)$ comparisons in the worst case through binary splitting. However, suppose we have a predictor h that estimates the position of each query element q. A natural algorithm would be:

(i) Probe position $h(q)$, if not found, determine which direction to search and probe positions $h(q) + 2^i$ or $h(q) - 2^i$ for increasing i until bracketing q.
(ii) Perform binary search on the identified range.

Let $\eta_q = |h(q) - t(q)|$ be the prediction error for q, where $t(q)$ is its true position. The cost of this algorithm is $O(\log \eta_q)$ comparisons. This simple example demonstrates the key principles: near-optimal performance with good predictions (approaching $O(1)$ as $\eta_q \to 0$) while maintaining worst-case ($O(\log n)$ even with arbitrarily poor predictions since $\eta_q \leq n$). The framework has been successfully applied across algorithmic domains. For online problems, such learning-augmented algorithms are typically characterized by two properties:

Definition 2.0.2 (α-Consistency)**.** A learning-augmented algorithm $\mathcal{A}$ is α-consistent if $\mathrm{CR}_{\mathcal{A}} \leq \alpha$ when the prediction error $\eta(h) = 0$.

Definition 2.0.3 (β-Robustness)**.** Algorithm $\mathcal{A}$ is β-robust if for any predictor h, $\mathrm{CR}_{\mathcal{A}} \leq \beta$.

Many fundamental algorithmic problems have been studied, including rent-or-buy(aka ski-rental) [67, 2, 78, 43], sorting [59, 30, 9], streaming algorithms [48, 35, 1,

74], mechanism design [85, 42, 66, 58, 61], graph problems [55, 33, 27, 28], packing [87, 49, 4], routing [53, 21, 45, 31], secretary problem [7, 36, 64, 41], data structures [54, 56, 38, 86, 61], scheduling [67, 3, 22, 8], clustering [65, 37] and etc.

2.1 Learning-augmented Caching

Two primary prediction models have emerged for caching. The first, introduced by Lykouris and Vassilvitskii (2021)[60], predicts the next occurrence time of each requested page. Formally, for a request sequence σ, the predictor h provides estimates $h(\sigma_t)$ of the true next occurrence time $y(\sigma_t)$. The prediction error is measured using the ℓ_1 metric:

$$\eta_{\ell_1}(h, \sigma) = \sum_t |h(\sigma_t) - y(\sigma_t)|$$

The second model, proposed by Antoniadis et al. (2023)[5], predicts the behavior of the optimal offline algorithm, which generalizes beyond caching to metrical task systems.

While there are variants such as weighted caching[51, 52] or caching with succinct predictions [50, 6], we focus on the first setting on the standard caching problem. Building upon the learning-augmented algorithmic framework, three seminal works established the theoretical foundations for augmenting caching algorithms with machine learned predictions. Here we examine their key algorithmic contributions and theoretical guarantees in detail.

The foundational work of Lykouris and Vassilvitskii (2021)[60] introduced PredictiveMarker, which modified the classical MARKER algorithm to incorporate predictions while maintaining worst-case guarantees. Their algorithm explicitly switches between trusting predictions and random marking based on eviction chain length.

PredictiveMarker operates in phases, with elements being marked upon arrival. When a cache miss occurs, if all elements are marked, a new phase begins with all elements unmarked. The key lies in how it handles cache misses within a phase. For each eviction chain (sequence of evictions triggered by a clean element - one that did not appear in the previous phase), PredictiveMarker follows BlindOracle on unmarked elements until the chain reaches length H_k, after which it reverts to random eviction of unmarked elements. This achieves a competitive ratio of $\min\{\, 2 + 4\sqrt{5\eta/\mathrm{OPT}}, 4H_k \,\}$.

Rohatgi (2020)[69] further refined this approach with two algorithms, LMarker and LNonMarker. LMarker simplifies PredictiveMarker by trusting predictions only once at the start of each eviction chain, achieving an improved asymptotic ratio of $O(1 + \min(\log(\eta/\mathrm{OPT}), \log k))$. LNonMarker introduces an additional random component - upon a cache miss, if the eviction chain is short, it follows BlindOracle, but if long, it evicts a random element from the entire cache rather than just unmarked elements. Through black-box combination with MARKER, this achieves another improved asymptotic ratio of $O\left(1 + \min\left(1, \frac{\eta/\mathrm{OPT}}{k}\right) \log k\right)$.

Wei (2020)[84] took a simpler yet more effective approach, showing that directly combining BlindOracle with a classical algorithm like LRU or MARKER in a black-

box manner achieves state-of-the-art bounds. Their key insight was that BlindOracle performs extremely well when predictions are accurate (η/OPT small), with competitive ratio $O(1+\frac{\eta}{k\cdot\text{OPT}})$. By using a "follow-the-leader" approach: simulating both algorithms and evicting any page that is not in the cache of the better performing one, they achieve $O(1+\min(\frac{\eta/\text{OPT}}{k}, \log k))$.

All of these algorithms embody the principle of dynamically switching between trusting predictions and classical eviction rules binarily. Among the three, the latest work of Wei [84]'s black-box combination achieves better theoretical bounds as well as empirical performance as shown by Chłędowski et al. (2021)[32].

2.2 Data-driven Algorithm Design

The challenge of selecting appropriate α parameters connects to the broader field of data-driven algorithm design[46, 10], where algorithm parameters are tuned using problem instances to achieve good expected performance. For a family of algorithms $\mathcal{A}$ parameterized by $\rho \in \mathcal{P}$, the traditional framework considers a distribution $\mathcal{D}$ over problem instances and aims to find parameters that perform well in expectation. Formally, given m training instances $x_1, \ldots, x_m$ drawn i.i.d. from $\mathcal{D}$, and a utility function $u(x, \rho)$ measuring the performance of algorithm A_ρ on instance x, the goal is to find:

$$\rho^* \in \arg\max_{\rho\in\mathcal{P}} \mathbb{E}_{x\sim\mathcal{D}}[u(x,\rho)]$$

For many algorithmic problems including caching, the utility functions $u(x, \rho)$ are piecewise structured with discontinuities. Recent work[12, 13] has shown that despite this non-smoothness, learning theoretic tools can provide strong generalization guarantees. The key insight is that while individual utility functions may be discontinuous, their family often has low "pseudo-dimension" - a complexity measure that bounds sample complexity. The problem of tune parameters using problem instances to achieve good expected performance has been studied in various contexts including learning sketching matrices in numerical linear algebra [23, 72, 71], learning optimal cooling schedules in simulated annealing [26], learning to link [11], branch [14] and Gomory mixed integer cuts [20] in MILP, learning rounding threshold in Integer Quadratic Programming [15], learning heuristic functions in pathfinding algorithms [73], learning to cluster [16], learning to regression [19, 17], learning revenue functions in mechanism design[63, 18, 29, 44], learning parameterized biology algorithms [13]) and etc.

In our setting, we face a more challenging online learning problem where data arrive sequentially and the distribution may change over time. Traditional multi-armed bandit or online convex optimization frameworks could be insufficient due to the fundamental non-convexity and discontinuities in our utility functions. Moreover, the distribution of requests may evolve over time, requiring adaptation to environment shifts. The online parameter learning framework of [77] considers this setting and shows that under a dispersion condition [12] on the sequence of utility functions, one can achieve

sublinear regret. Specifically, for a sequence of functions that are β-dispersed - meaning that for any ball of radius ϵ, at most $O(\epsilon T)$ functions are non-Lipschitz in that ball for $\epsilon \geq T^{-\beta}$ - they show regret bounds of $O\left(\sqrt{sdT \log T} + sT^{1-\beta}\right)$ where d is the dimension of parameter space and s is the number of environment shifts.

While this framework provides important theoretical foundations, direct application to our caching parameter tuning problem is highly inefficient - their approach requires maintaining weights over the entire parameter space $\mathcal{P}$, making it computationally intractable. Their analysis assumes sudden arbitrary environment shifts and general function classes while our setting exhibits more favorable properties: the cache hit rate typically demonstrates approximate unimodality, and distribution shifts tend to be gradual rather than adversarial. This suggests opportunities for more efficient solutions.

2.3 Gaussian Process Regression

We first introduce Gaussian process (GP) regression for modeling and optimizing unknown functions. A Gaussian process [68] provides a probabilistic model of an unknown function $f : \mathcal{X} \to \mathbb{R}$, characterized by a mean function $\mu(\cdot)$ and covariance (kernel) function $k(\cdot, \cdot)$. Given observations $\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n$ where $y_i = f(x_i) + \eta_i$ and $\eta_i \sim \mathcal{N}(0, \sigma^2)$, the posterior distribution at any point x is Gaussian with mean and variance:

$$\mu_n(x) = k_n(x)^T (K_n + \sigma^2 I)^{-1} y_n$$
$$\sigma_n^2(x) = k(x, x) - k_n(x)^T (K_n + \sigma^2 I)^{-1} k_n(x)$$

where $k_n(x) = [k(x_1, x), \ldots, k(x_n, x)]^T$ and $K_n = [k(x_i, x_j)]_{i,j=1}^n$ is the kernel matrix. Common kernel choices include the squared exponential (SE) kernel:

$$k_{SE}(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2l^2}\right)$$

and the Matérn kernel with smoothness parameter ν:

$$k_\nu(x, x') = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\sqrt{2\nu}\frac{\|x - x'\|}{l}\right)^\nu K_\nu \left(\sqrt{2\nu}\frac{\|x - x'\|}{l}\right)$$

where l is the length-scale parameter and K_ν is the modified Bessel function.

2.3.1 Acquisition Functions

To balance exploration and exploitation in optimization, acquisition functions are used to determine the next query point. Two popular choices are Expected Improvement (EI) and Upper Confidence Bound (UCB).

Expected Improvement
EI selects points that maximize the expected improvement over the current best observation $y^* = \max_{i \leq n} y_i$:

$$EI(x) = \mathbb{E}[\max(f(x) - y^*, 0)] = \sigma_n(x)(z\Phi(z) + \phi(z))$$

where $z = (\mu_n(x) - y^*)/\sigma_n(x)$, and Φ, ϕ are the standard normal CDF and PDF respectively. This has a convenient analytical form enabling efficient optimization.

Upper Confidence Bound
GP-UCB selects points that maximize:

$$UCB(x) = \mu_n(x) + \beta_n^{1/2}\sigma_n(x)$$

where β_n is a parameter controlling exploration. While EI naturally balances exploration-exploitation, UCB requires appropriate tuning of β_n but often admits simpler theoretical analysis.

2.3.2 Theoretical Analysis

A key concept in the theoretical analysis of GP-based optimization is the maximum information gain[80]:

$$\gamma_T = \max_{A:|A|=T} \frac{1}{2} \log |I + \sigma^{-2} K_A|$$

which captures the maximum reduction in uncertainty about f after T observations. This quantity appears in regret bounds and has been shown to be sublinear in T for common kernels[80]:

- Linear: $\gamma_T = O(d \log T)$
- SE: $\gamma_T = O((\log T)^{d+1})$
- Matérn: $\gamma_T = O(T^{\frac{d(d+1)}{2\nu + d(d+1)}} \log T)$

[80] showed for functions in the RKHS $\mathcal{H}_k$, under certain conditions, GP-UCB achieves regret:

$$R_T = O\left(\sqrt{T\beta_T\gamma_T}\right)$$

with high probability using appropriate β_T.

These results assume a stationary environment where f does not change over time. For non-stationary settings, [88] proposed using a sliding window approach, where only the most recent w observations are used for inference. Their algorithm SW-GP-UCB achieves regret:

$$R_T = O\left(\sqrt{\gamma_w} w^{3/2} P_T + \beta_T T \sqrt{\frac{\gamma_w}{w}}\right)$$

with high probability, where $\sum_{t=1}^{T-1} \|f_{t+1} - f_t\|_{\mathcal{H}} \leq P_T$. With optimal choice of window size w, this yields:

$$R_T = O\left(\gamma_T^{7/8}(1+P_T)^{1/4}T^{3/4}\right)$$

An alternative approach[34] uses weighted observations, assigning higher weights to recent data points. This introduces weighted variants of the posterior mean and variance:

$$\tilde{\mu}_n(x) = \tilde{k}_n(x)^T(\tilde{K}_n + \lambda_n I)^{-1}\tilde{y}_n$$
$$\tilde{\sigma}_n^2(x) = k(x,x) - \tilde{k}_n(x)^T(\tilde{K}_n + \lambda_n I)^{-1}\tilde{k}_n(x)$$

where $\tilde{K}_n = WK_nW^T$, $\tilde{k}_n(x) = Wk_n(x)$, and $W = \text{diag}(\sqrt{w_1}, \ldots, \sqrt{w_n})$ contains the observation weights. While this approach can achieve similar theoretical guarantees as the sliding window method, it becomes computationally expensive as the number of observations grows, since all historical data points must be maintained and weighted.

For caching applications where computational efficiency is critical, we adopt the sliding window approach.

Chapter 3

Proposed Method

3.1 Weighted Integration

Consider a simple way to integrate ML predictions with classical caching heuristics, focusing specifically on LRU. Let x be a page in cache, $h(x)$ denote the ML-predicted reuse distance, and $l(x)$ denote the most recent timestep at which x was accessed. The classical LRU eviction policy selects:

$$\arg\min_x l(x) = \arg\max_x (t - l(x))$$

At time t, for a cached page x, we assume $t - l(x)$ serves as an estimator for $n(x) - t$, where $n(x)$ denotes the next request time of x. This estimator exhibits an interesting unbiasedness property:

Lemma 3.1.1 (LRU Unbiasedness(Linear))**.** For any page x, the average error of using $t - l(x)$ to estimate $n(x) - t$ over the interval $[l(x) + 1, n(x) - 1]$ is 0.

Proof.

$$\begin{aligned}\sum_{t=l(x)+1}^{n(x)-1} ((t - l(x)) - (n(x) - t)) &= \sum_{t=l(x)+1}^{n(x)-1} (t - l(x)) - \sum_{t=l(x)+1}^{n(x)-1} (n(x) - t)) \\ &= \sum_{i=1}^{n(x)-l(x)-1} i - \sum_{j=1}^{n(x)-l(x)-1} j = 0\end{aligned}$$

□

Thus, $t - l(x)$ serves as a proxy of unbiased estimator of the reuse distance. For the machine learning predictor h(x), we assume it provides another unbiased estimate of the reuse distance. Given two unbiased estimators, their combination can potentially achieve lower variance - a classical result in forecasting and model averaging, dating back to Bates and Granger (1969)[24]. Specifically, for estimators d_1 and d_2 with variances σ_1^2 and σ_2 and correlation ρ, the variance of a weighted combination $d_3 = \alpha d_1 + (1 - \alpha) d_2$is:

$$\mathbb{V}[d_3] = \alpha^2\sigma_1^2 + (1 - \alpha)^2\sigma_2^2 + 2\alpha(1 - \alpha)\rho\sigma_1\sigma_2$$

When the estimators are not perfectly correlated ($\rho < 1$), there exists a unique α that minimizes this variance:

$$\alpha^* = \frac{\sigma_2^2 - \rho\sigma_1\sigma_2}{\sigma_1^2 + \sigma_2^2 - 2\rho\sigma_1\sigma_2}$$

By taking $\alpha = \alpha^*$, it can be shown that $\text{Var}(d_3) \leq \min\{\sigma_1^2, \sigma_2^2\}$.

The existence of a unique variance minimizer aligns with our experimental observations that the cache hit rate as a function of α exhibits approximate unimodality - there is typically a single region of α that yields the highest performance. This theoretical connection helps explain why blending ML predictions with LRU's implicit distance estimates proves effective in practice.

Note that we need to choose proper $\alpha(x)$ for each request x. We first start with the simplest approach - use a constant α for each request. However, the scale of reuse distances can vary dramatically across different requests, making constant variance assumptions unrealistic. We address this through a log transformation and model as follows:

$$\begin{aligned}
\tilde{d}(x) &= \log d(x) \\
\tilde{h}(x) &= \log h(x) = \tilde{d}(x) + \epsilon_1(x) \\
\tilde{g}(x) &= \log(t - l(x)) = \tilde{d}(x) + \epsilon_2(x)
\end{aligned}$$

where $\epsilon_1(x)$ and $\epsilon_2(x)$ are zero-mean error terms with constant variances σ_1^2 and σ_2^2 respectively. Furthermore, it aligns with how modern ML models are typically trained, using log distance as the prediction target. In addtion, the unbiasedness property still holds:

Lemma 3.1.2 (LRU Unbiasedness(Log))**.** For any page x, the average error of using $\log(t - l(x))$ to estimate $\log(n(x) - t)$ over the interval $[l(x) + 1, n(x) - 1]$ is 0.

Proof. Similar to the linear case,

$$\sum_{t=l(x)+1}^{n(x)-1} (\log(t - l(x)) - \log(n(x) - t)) = \sum_{i=1}^{n(x)-l(x)-1} \log i - \sum_{j=1}^{n(x)-l(x)-1} \log j = 0.$$

□

This leads to our proposed eviction rule:

$$\arg\max_{x \in S} (\log h(x) + \alpha \log(t - l(x)))$$

3.2 Some Analysis

We begin by recalling Wei's bound for BlindOracle, which serves as the foundation for our analysis:

Theorem 3.2.1 ([84])**.** BlindOracle obtains a competitive ratio of:

$$\min\left(1+2\frac{\eta}{\mathrm{OPT}}, 4+\frac{4}{k-1}\frac{\eta}{\mathrm{OPT}}\right)$$

where η is the ℓ_1 prediction error and OPT is the optimal offline cost.

This result, combined with the combiner framework from Wei's work, yields the following, which establishes the similar worst-case guarantees of our proposed approach:

Theorem 3.2.2 ([84])**.** There exists a deterministic algorithm for learning-augmented online caching that achieves a competitive ratio of

$$2\min\left(\min\left(1+2\frac{\eta}{\mathrm{OPT}}, 2+\frac{4}{k-1}\frac{\eta}{\mathrm{OPT}}\right), k\right).$$

Such algorithm achieves 2-consistency and $2k$-robustness.

Theorem 3.2.3 ([84])**.** There exists a randomized algorithm for learning-augmented online caching that achieves a competitive ratio of

$$(1+\varepsilon)\min\left(\min\left(1+2\frac{\eta}{\mathrm{OPT}}, 2+\frac{4}{k-1}\frac{\eta}{\mathrm{OPT}}\right), H_k\right)$$

for any $\varepsilon \in (0, 1/4)$. Such algorithm achieves $(1+\epsilon)$-consistency and $(1+\epsilon)H_k$-robustness.

3.2.1 Connection to ℓ_1 Error

To connect this variance-based analysis to the competitive ratio bounds in prior work, we can use the following relationships between variance and ℓ_1 error:

Proposition 3.2.4 (Linear Estimators)**.** For any unbiased estimator $\hat{d}$ of d with variance σ^2:

$$\mathbb{E}[|d-\hat{d}|] \le \sqrt{\mathbb{E}\left[(d-\hat{d})^2\right]} = \sigma$$

Specifically when $\hat{d}$ follows a normal distribution,

$$\mathbb{E}[|d-\hat{d}|] = \sigma\sqrt{\frac{2}{\pi}}$$

Proof. First inequality follows from Jensen's inequality. The second is a classical result. □

Proposition 3.2.5 (Log Estimators)**.** For any unbiased estimator $\log\hat{d}$ of $\log d$ with variance σ^2, $\mathbb{E}[|d-\hat{d}|]$ is increasing with σ. Furthermore, when $\log\hat{d}$ follows a normal distribution,

$$\mathbb{E}[|d-\hat{d}|] = e^{\sigma^2/2}(2\Phi(\sigma)-1)d$$

where $\Phi(\cdot)$ is the standard normal CDF.

Proof. Let $Z = \log \hat{d} - \log d$, then

$$\begin{aligned}\mathbb{E}[|\hat{d} - d|] &= \mathbb{E}\left[d\left|\frac{\hat{d}}{d} - 1\right|\right] \\ &= \mathbb{E}\left[\left|e^{\log(\hat{d}) - \log d} - 1\right|\right] d \\ &= \mathbb{E}\left[\left|e^{Z} - 1\right|\right] d\end{aligned}$$

Let $Y = Z/\sigma$, then since

$$\frac{d}{d\sigma}|e^{\sigma y} - 1| = |y|e^{\sigma y} \geq 0,$$

therefore $\mathbb{E}[|\hat{d}(x) - d(x)|]$ is increasing with σ.

When Z follows a normal distribution,

$$\begin{aligned}\mathbb{E}[|e^{Z} - 1|] &= \int_{-\infty}^{0}(1 - e^{z})f(z)dz + \int_{0}^{\infty}(e^{z} - 1)f(z)dz \\ &= 2\mathbb{E}[e^{Z}\mathbf{1}\{Z > 0\}] - \mathbb{E}[e^{Z}] \\ &= 2e^{\sigma^2/2}\Phi(\sigma) - e^{\sigma^2/2}.\end{aligned}$$

In the last equality, the second

□

These relationships allow us to bound the expected ℓ_1 error in terms of our variance analysis.

3.2.2 Refined Bounds

Previous bounds involve ℓ_1 errors that can be $O(n^2)$ where n is the sequence length. We propose two bounds which can reduce $O(n^2)$ to $O(n)$ and $O(nk)$ at the cost of being less tractable.

First, we formalize the notion of miseviction:

Definition 3.2.1 (Miseviction)**.** At time t where algorithm $\mathcal{A}$ must evict some page, let y be the page in cache whose actual next arrival is furthest in the future. If $\mathcal{A}$ evicts any page other than y, we say it commits a miseviction.

This leads to a tighter analysis:

Proposition 3.2.6. Let $M_{\mathcal{A}}$ be the total number of misevictions incurred by $\mathcal{A}$ over sequence σ. Then

$$\text{cost}_{\mathcal{A}}(\sigma) \;\leq\; \text{OPT}(\sigma) \;+\; M_{\mathcal{A}}(\sigma)$$

Proof. This follows directly from Proposition 3.1 in Wei (2020)[84] by observing that $\Delta M = 1$ occurs if and only if $\mathcal{A}$ misevicts. □

Corollary. Let p be the ratio of misevictions by algorithm $\mathcal{A}$. Then:

$$\mathrm{CR}_{\mathcal{A}} = \frac{\mathrm{ALG}_{\mathcal{A}}}{\mathrm{OPT}} \leq \frac{1}{1-p}$$

Proof. Substituting $M_{\mathcal{A}} = p \cdot \mathrm{ALG}_{\mathcal{A}}$ into Proposition 3.2.6 yields the result. □

To analyze our weighted combination approach in this framework, we examine how variance affects miseviction probability:

Proposition 3.2.7. Let $d_0 > d_1 > ... > d_{k-1}$ be true reuse distances and $\hat{d}_i = d_i + \epsilon_i$ where ϵ_i are i.i.d zero-mean noise with variance σ^2. Then $\mathbb{P}(\hat{i} \neq 0)$ is monotonically increasing in σ, where $\hat{i} = \arg\max_i \hat{d}_i$.

Proof. Let $t_i = d_0 - d_i > 0$. Then we have

$$\{\hat{i} \neq 0\} = \bigcup_{i=1}^{k-1} \{\hat{d}_i > \hat{d}_0\} = \bigcup_{i=1}^{k-1} \{\epsilon_i - \epsilon_0 > t_i\}.$$

Since ϵ_i have variance σ^2, we write $\epsilon_i = \sigma Z_i$ where Z_i are i.i.d. with unit variance. Let $T_i = Z_i - Z_0$. Then:

$$\{\epsilon_i - \epsilon_0 > t_i\} = \{T_i > t_i/\sigma\}$$

Therefore,

$$\mathbb{P}(\hat{i} = 0) = \mathbb{P}\left(\bigcap_{i=1}^{k-1} \{T_i \leq t_i/\sigma\}\right)$$

Now for any $\sigma_1 < \sigma_2$, $\frac{t_i}{\sigma_2} < \frac{t_i}{\sigma_1}$ for any i, which implies

$$\bigcap_{i=1}^{k-1} \{T_i \leq t_i/\sigma_2\} \subseteq \bigcap_{i=1}^{k-1} \{T_i \leq t_i/\sigma_1\}.$$

Taking probabilities,

$$\mathbb{P}(\hat{i} = 0)|_{\sigma=\sigma_2} \leq \mathbb{P}(\hat{i} = 0)|_{\sigma=\sigma_1}.$$

Therefore $\mathbb{P}(\hat{i} \neq 0)$ is monotonically increasing in σ. □

Now let's move to the second refined bound.

Definition 3.2.2. For a cache fault occurring at time t, let the requested page be p, define the miseviction index i_t as:

$$i_t = |\{l \mid \hat{d}_p \geq \hat{d}_l\}|$$

where $\hat{d}$ is the estimated reuse distance of the requested page and $\hat{d}_l$ are the estimates for pages in cache.

Proposition 3.2.8.

$$\mathrm{CR}_{\mathcal{A}} \leq 2 + \frac{2}{(k-1)\mathrm{OPT}} \sum_{t=1}^{\mathrm{ALG}_{\mathcal{A}}} i_t$$

Proof. Following the case analysis of Proposition 3.3 in Wei [84], we observe that the inequality still holds when replacing the number of inversions with the miseviction index defined above:

$$(k-1)\Delta\mathrm{ALG}_{\mathcal{A}} + \Delta\Phi \leq 2(k-1)\Delta\mathrm{OPT} + 2\Delta\left(\sum_{t=1}^{\mathrm{ALG}_{\mathcal{A}}} i_t\right)$$

holds at each time step. Summing over all requests yields the result. □

3.3 Online Parameter Tuning

While the previous analysis establishes that combining ML predictions with LRU through an appropriate weighting parameter α can reduce prediction variance and improve cache hit rates, in practice, we need to learn this parameter online as the input sequence arrives. As stated previously, determining this parameter online presents several challenges. First, the optimal α may vary across different workloads and even within a single sequence as access patterns evolve. Second, while the performance curve exhibits nice structure when evaluated on complete sequences, limited data during online learning leads to high-variance estimates. Finally, for practical applications, we cannot exhaustively sample all α values at all times. Below, we propose two approaches.

3.3.1 Ternary/Golden-Section Search over α

Our first approach leverages the empirically observed approximate unimodality of cache hit rate with respect to α. We consider periodically optimizing α using golden-section search over a sliding window of recent requests. Let W denote the window size and M the update period. Let $f_t(\alpha)$ denote the cache hit rate achieved by weight α on the request sequence from time $t - W$ to t. After every M requests,

(i) Run golden-section search on the last W requests to approximate

$$\alpha_t = \underset{\alpha \in [\alpha_{\min}, \alpha_{\max}]}{\arg\max} \; f_t(\alpha)$$

(ii) Update α to the α_t

Over the whole sequence, α are updated $O(n/M)$ times while finding α_t via simulation costs $O(W \log(\frac{1}{\epsilon}))$, therefore the total computational complexity is $O(n + nW\log(\frac{1}{\epsilon})/M)$ where ϵ is the accuracy parameter of golden-section search.

Algorithm 1 Weighted Integrated Caching with Periodic Golden-Section Search

Require: Request sequence σ, cache size k, period M, window size W, tolerance ϵ, $\alpha_{\min}$, $\alpha_{\max}$

Initialize $\alpha \leftarrow 1.0$
for $t \leftarrow 1$ to $|\sigma|$ **do**
  if $t \bmod M = 0$ **then** ▷ Time to update α
    window $\leftarrow \sigma[\max\{1, t - W\} : t]$
    $a \leftarrow \alpha_{\min}$, $b \leftarrow \alpha_{\max}$
    while $b - a > \epsilon$ **do**
      $c \leftarrow b - \phi(b - a)$ ▷ Golden ratio points
      $d \leftarrow a + \phi(b - a)$
      $f_c \leftarrow$ simulate_cache(window, c)
      $f_d \leftarrow$ simulate_cache(window, d)
      if $f_c > f_d$ **then**
        $b \leftarrow d$
      else
        $a \leftarrow c$
      end if
    end while
    $\alpha \leftarrow (a + b)/2$ ▷ Update α
  end if
  if $\sigma_t \notin S$ **then**
    $x^* \leftarrow \arg\max_{x \in S} (\log h(x) + \alpha \log(t - l(x)))$ ▷ evict based on current α
    $S \leftarrow S \setminus \{x^*\} \cup \{\sigma_t\}$
  end if
end for

3.3.2 Bayesian Optimization with a Sliding Window

While golden-section search assumes unimodality and could converge to local optima, Bayesian Optimization (BO) methods can more gracefully handle noise due to limited data as well as adapt to changing functions. Our second approach is to model the true performance function f_t using a Gaussian Process (GP) with a sliding window of observations. The sliding window keeps only the most recent K observations $(\alpha_i, f_{t_i}(\alpha_i))_{i=1}^{K}$, allowing the model to adapt as access patterns evolve while maintaining $O(K^3)$ complexity for GP inference rather than growing unbounded.

Algorithm 2 Gaussian Process Based Adaptive Caching

Require: Request sequence σ, cache size k, window size K, GP hyperparameters θ

Initialize $\alpha \leftarrow 1.0$.
Initialize $\mathcal{D} \leftarrow \emptyset$ ▷ GP training data
Initialize GP prior with kernel k_θ
for $t \leftarrow 1$ to $|\sigma|$ **do**
  if $t \bmod M = 0$ **then** ▷ Time to update GP model
    $y \leftarrow$ compute_window_hitrate$(t - W : t)$
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\alpha, y)\}$
    if $|\mathcal{D}| > K$ **then** ▷ Maintain sliding window
      $\mathcal{D} \leftarrow$ most recent K points
    end if
    Update GP posterior using $\mathcal{D}$
    $\alpha \leftarrow \arg\max_{\alpha} \text{Acq}(\alpha)$ ▷ Optimizing acquisition functions(EI or UCB)
  end if
  if $\sigma_t \notin S$ **then**
    $x^* \leftarrow \arg\max_{x \in S} (\log h(x) + \alpha \log(t - l(x)))$ ▷ evict based on current α
    $S \leftarrow S \setminus \{x^*\} \cup \{\sigma_t\}$
  end if
end for

The computational complexity: for every M requests,

- GP inference: $O(K^3)$ per update for exact inference with K points
- Acquisition function optimization: $O(K^2C)$ for using C candidates to evaluate

The total complexity is thus $O(n + \frac{n}{M}(K^3 + K^2C + W))$. With $C = K = O(n^{\frac{1}{6}})$, $W = M = O(n^{\frac{1}{2}})$, we obtain $O(n)$.

Assume each f_t lies in an RKHS and the total variations of f_t is bounded as $\sum_{t=1}^{T-1} \|f_{t+1} - f_t\|_{\mathcal{H}} \leq P_T$. Then as discussed in Section 2.3.2, selecting UCB as the acquisition function with appropriate window size K achieves a regret of $R_T = O\left(\gamma_T^{7/8}(1 + P_T)^{1/4}T^{3/4}\right)$ with high probability.

Chapter 4

Experiment

We conduct extensive experiments to evaluate our proposed weighted combination method with online parameter tuning methods on real-world caching workloads.

4.1 Experimental Setup

4.1.1 Datasets

Following prior work [32], we evaluate on the benchmark datasets from the 2nd Cache Replacement Championship (CRC-2) [81]. These datasets consist of real-world memory access traces from the SPEC CPU2006 benchmark suite[47], representing diverse workload patterns from different applications. For each trace, following [32], we subsample by selecting 64 out of 2048 sets and filtering accesses to those sets. We split each resulting trace into training (80%), validation (10%), and test (10%) sets. The datasets vary significantly in size and characteristics, which allows us to evaluate across different scenarios. For example, mcf shows highly irregular access patterns due to pointer-chasing behavior, while libquantum has streaming access patterns. Also, the size ranges from 69,120 requests (xalanc) to 2,965,504 requests (mcf).

	astar	bwaves	bzip	cactusadm	gems	lbm	leslie3d	libq	mcf	milc	omnetpp	sphinx3	xalanc
Size	1,154,048	570,368	167,680	221,952	723,456	782,080	716,032	579,840	2,965,504	556,800	555,520	328,704	69,120
Cache hits (%)													
OPT	37.4	4.9	80.8	33.7	12.7	24.8	30.9	5.3	44.6	1.4	42.4	74.8	56.9
RANDOM	8.3	0.2	56.5	4.5	3.9	2.2	9.5	0.0	20.5	0.0	17.6	52.8	36.8
LRU	4.0	0.0	63.8	0.0	2.9	0.0	9.5	0.0	27.1	0.0	20.4	12.7	45.4
MARKER	4.7	0.0	62.7	1.3	4.1	0.0	9.3	0.0	24.9	0.0	20.3	42.2	43.5
PARROT-REUSE	29.0	0.1	57.9	21.8	0.4	0.5	4.9	5.3	32.5	1.1	11.9	67.6	37.3

Table 4.1: Characteristics of the dataset[32]. The first row shows the total sizes of all used datasets. These sizes are later split into train/valid/test sets with 80%/10%/10% splits. Further rows display the cache hit rates of pure (non-learning-augmented) algorithms, illustrating varying difficulties of the datasets.

4.1.2 ML model

We implement our methods in Python, building upon the open-source code of prior work[32]. We use a cache size of $k = 16$ pages for all experiments, matching prior work. The Parrot model[57] is used for reuse distance predictions. The model takes as input the last $H = 30$ requested memory addresses and the k addresses currently in the cache. Each address is embedded into a 32-dimensional vector using a byte-level embedding network to reduce memory overhead. Specifically, each byte of the address is embedded separately using a small linear layer, and the byte embeddings are concatenated and passed through another linear layer to produce the final address embedding. The sequence of recent access embeddings is processed by an LSTM with 128 hidden units. Then a combination of attention mechanisms from the Transformer [82] and BiDAF [76] architectures is used to compute cache line scores. For each line in the cache, a context vector is generated by attending over the LSTM hidden states using the line's embedding as the query. The context vectors are passed through a final linear layer to predict reuse distances. The model is trained to minimize mean squared error on log-transformed reuse distances. We use the Adam optimizer with learning rate 0.001 and batch size 32. We disable DAgger (for learning optimal policy) and only use the reuse distance predictions. Training runs for 20,000 steps with checkpoints every 5,000 steps, selecting the model with the highesthesthest validation hit rate.

4.1.3 Baselines and Implementation Details

Our experimental evaluation provides a comprehensive comparison against all major learning-augmented caching algorithms from prior work. For meaningful cross-dataset comparisons where absolute hit rate may vary significantly, we normalize performance using the LRU-normalized empirical competitive ratio, i.e.,

$$\frac{\mathrm{CR}_{ALG} - 1}{\mathrm{CR}_{\mathrm{LRU}} - 1}$$

This metric normalizes performance such that $\mathrm{CR}_{\mathrm{OPT}} = 0$ and $\mathrm{CR}_{\mathrm{LRU}} = 1$, where lower values indicate that the algorithms perform closer to the offline optimal.

We evaluate against the following algorithms (following prior work's naming convention for consistency):

- LRU and MARKER: Two classical baselines
- PARROT-REUSE: which is BlindOracle - Direct use of ML predictions to emulate Belády's optimal policy
- PredictiveMarker: The first learning-augmented caching algorithm
- LMarker and LNonMarker: Improved variants with better theoretical bounds
- $\mathrm{BlindOracle}^{\mathrm{D}}$/$\mathrm{BlindOracle}^{\mathrm{R}}$: State-of-the-art approaches that combine classical algorithms with pure prediction following using either deterministic (D) or randomized (R) combiners.

For online parameter tuning, we implement two methods:

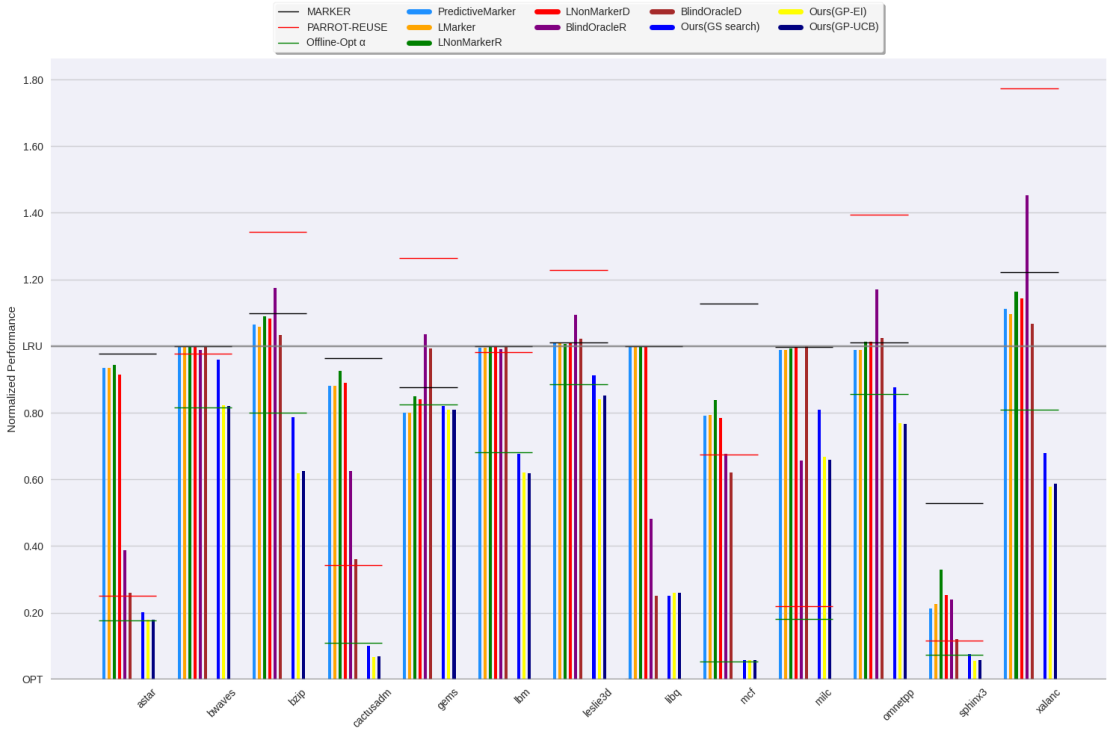

Fig. 4.1: Normalized performance comparison across SPEC CPU2006 benchmarks.

(i) Golden-Section(GS) Search using $M = W = \lfloor \sqrt{n} \rfloor$ where n is sequence length
(ii) Gaussian Process using Matérn kernel with sliding window where its size $K = 30$, $W = \lfloor \sqrt{n} \rfloor$ and $M = \lfloor W/5 \rfloor$. We evaluate both Expected Improvement (EI) and Upper Confidence Bound (UCB) acquisition functions.

The window parameter selection reflects fundamental tradeoffs in online learning: smaller windows enable rapid adaptation but suffer from estimation variance, while larger windows provide statistical stability at the cost of slower adaptation to distribution shifts. We found that scaling with $\sqrt{n}$ - setting $M = W = O(\sqrt{n})$ provides a good balance - adjusting to dataset size while maintaining reasonable computational overhead.

4.2 Results and Analysis

4.2.1 State-of-the-Art Performance Among Baselines

Figure 4.1 shows the normalized performance across all benchmarks. The horizontal gray line at 1.0 represents LRU's performance, while 0.0 represents OPT.

Our results align with previous findings showing BlindOracle^{D} as the strongest baseline. Notably, MARKER performs comparably to LRU across most benchmarks,

validating our choice of LRU for maintaining fair comparisons. The deterministic combiner (BlindOracle^{D}) outperforms its randomized counterpart (BlindOracle^{R}) despite theoretical results, which demonstrates how real-world access patterns permit more aggressive exploitation than worst-case analysis suggests.

Interestingly, BlindOracle^{D} occasionally outperforms both of its components (MARKER and PARROT-REUSE), This result stems from evolving access patterns - switching between strategies can provide benefits when patterns change. However, this binary switching still fundamentally limits performance compared to true integration of signals.

4.2.2 Consistent Improvements from Weighted Integration

Our weighted integration approaches (rightmost 3 columns for each dataset) with online parameter tuning reliably outperform both components (LRU and PARROT-REUSE). The green horizontal lines indicate performance achieved using the best fixed α found through extensive offline simulation, demonstrating that our online tuning approaches often achieve near-optimal weighting. The Gaussian Process approaches (both EI and UCB variants) outperform Golden-Section Search. We find no significant performance difference between EI and UCB acquisition functions, suggesting both effectively balance exploration and exploitation in this setting.

4.2.3 Parameter Tuning Analysis

(i) Window size was tuned across $\{10, 20, 30, 40\}$ and we found that 30 offers the best result, which balances between sufficient data for reliable GP fitting and quick adaptation to changing access patterns.
(ii) Kernel Choice: The Matérn kernel slightly outperforms SE kernel, likely due to its better handling of our performance function that involves large local variations.

4.2.4 Further Discussion

Figure 4.2 provides detailed insight into the parameter learning dynamics. The left panel illustrates the GP-based approach's navigation of the performance landscape, showing quick convergence to good regions and stable tracking of optimal α values. The discrepancy between observed performance (red points) and the true current performance function (black dashed line) illustrates the fundamental challenge of online tuning - the objective function itself evolves as the sliding window shifts.

The GP posterior successfully captures both global trends and local structure in the performance function. The confidence intervals (shaded region) appropriately expand in sparsely sampled regions while remaining tight where observations cluster, enabling effective uncertainty-aware exploration. The right panel contrasts convergence behavior between GP-UCB and Golden-Section search. While computationally efficient, the Golden-Section approach exhibits vulnerability to local optima, converging

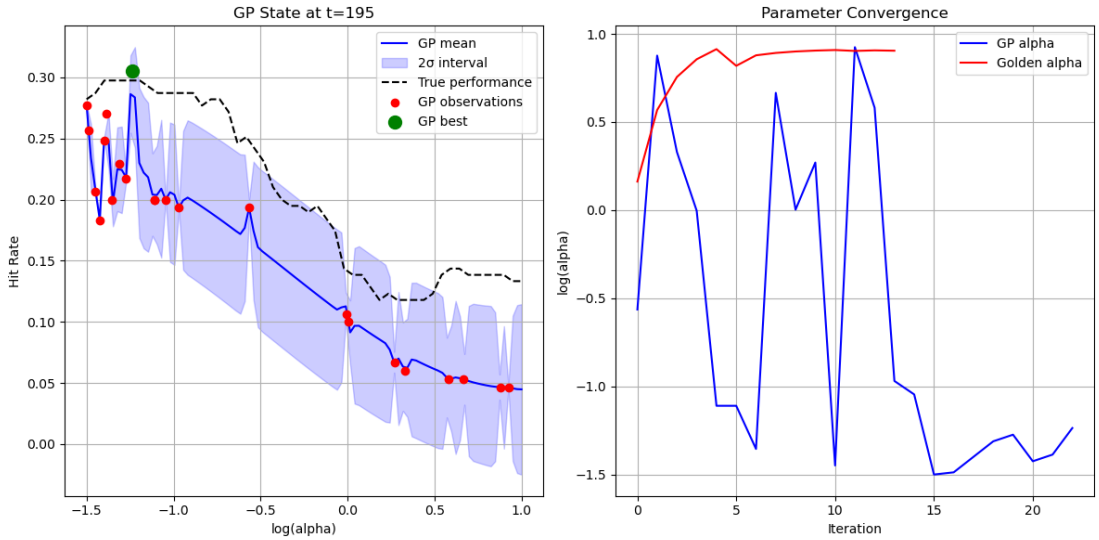

Fig. 4.2: Temporal dynamics of parameter tuning on the astar dataset. Left: GP posterior at $t = 195$ showing mean prediction (blue line), confidence intervals (shaded region), and observations (red points). The black dashed line represents the true performance function evaluated on the most recent window, which differs from older observations (red points) due to the sliding window approach. The green point indicates the current best α. Right: Parameter convergence comparison between GP-UCB (blue) and Golden-Section search (red). Golden-Section search converges to a suboptimal local maximum around $\log(\alpha) \approx 0.9$. Note that iteration counts differ since Golden-Section search is done on the most recent window until converged while GP shows the history of evaluated points over multiple recent windows.

to a suboptimal local maximum near $\log(\alpha) \approx 0.9$.

Chapter 5

Conclusion

This thesis introduced a novel approach to learning-augmented caching that fundamentally rethinks the integration of classical algorithms with machine learning predictions. Rather than viewing these components as competing alternatives to switch between, we demonstrated both theoretically and empirically that treating them as complementary signals to be combined yields superior performance. Our weighted integration framework maintains worst-case guarantees while significantly improving empirical performance over prior approaches. The theoretical analysis connecting variance reduction to competitive ratios provides insight into why such integration succeeds, while our online parameter tuning methods enable practical deployment.

Several promising directions remain for future investigation. First, our current approach uses a single weighting parameter across all requests. A next step is to more directly leverage modern uncertainty quantification methods in machine learning to obtain uncertainty estimates for each prediction. Rather than using a fixed weighting parameter α, this would enable request-specific adaptation based on the model's confidence in each prediction. Techniques like conformal prediction[83] could provide rigorous uncertainty estimates that would allow the algorithm to smoothly interpolate between following predictions and falling back to classical rules.

On the theoretical side, while we established worst-case guarantees matching prior work, our empirical results suggest these bounds may be quite loose. The refined analysis examining miseviction probabilities and indices takes initial steps toward tighter bounds. However, fully characterizing how prediction errors impact competitive ratios remains an open challenge. Finally, our weighted integration framework could be extended beyond the current setup of combining two signals (ML predictions and LRU). One direction is to develop methods for integrating multiple predictors, multiple classical heuristics and even the aforementioned uncertainty quantities while maintaining theoretical guarantees. More broadly, similar integration approaches could be valuable in other learning-augmented algorithms beyond caching.

Bibliography

[1] Anders Aamand et al. "Improved Frequency Estimation Algorithms with and without Predictions". In: *Thirty-seventh Conference on Neural Information Processing Systems*. 2023. URL: https://openreview.net/forum?id=0VcvYQ3uPh.

[2] Keerti Anand, Rong Ge, and Debmalya Panigrahi. "Customizing ML predictions for online algorithms". In: *Proceedings of the 37th International Conference on Machine Learning*. ICML'20. JMLR.org, 2020.

[3] Spyros Angelopoulos and Shahin Kamali. "Contract Scheduling With Predictions". In: *Proceedings of the AAAI Conference on Artificial Intelligence* 35.13 (May 2021), pp. 11726–11733. DOI: 10.1609/aaai.v35i13.17394. URL: https://ojs.aaai.org/index.php/AAAI/article/view/17394.

[4] Spyros Angelopoulos, Shahin Kamali, and Kimia Shadkami. "Online Bin Packing with Predictions". In: *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*. Ed. by Lud De Raedt. Main Track. International Joint Conferences on Artificial Intelligence Organization, July 2022, pp. 4574–4580. DOI: 10.24963/ijcai.2022/635. URL: https://doi.org/10.24963/ijcai.2022/635.

[5] Antonios Antoniadis et al. "Online Metric Algorithms with Untrusted Predictions". In: *ACM Trans. Algorithms* 19.2 (Apr. 2023). ISSN: 1549-6325. DOI: 10.1145/3582689. URL: https://doi.org/10.1145/3582689.

[6] Antonios Antoniadis et al. "Paging with Succinct Predictions". In: *Proceedings of the 40th International Conference on Machine Learning*. Ed. by Andreas Krause et al. Vol. 202. Proceedings of Machine Learning Research. PMLR, 23–29 Jul 2023, pp. 952–968. URL: https://proceedings.mlr.press/v202/antoniadis23a.html.

[7] Antonios Antoniadis et al. "Secretary and online matching problems with machine learned advice". In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 7933–7944.

[8] Yossi Azar, Stefano Leonardi, and Noam Touitou. "Flow time scheduling with uncertain processing time". In: *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*. STOC 2021. Virtual, Italy: Association for Computing Machinery, 2021, pp. 1070–1080. ISBN: 9781450380539. DOI: 10.1145/3406325.3451023. URL: https://doi.org/10.1145/3406325.3451023.

[9] Xingjian Bai and Christian Coester. "Sorting with Predictions". In: *Thirty-seventh Conference on Neural Information Processing Systems*. 2023. URL: https://openreview.net/forum?id=Qv7rWR9JWa.

[10] Maria-Florina Balcan. "Data-Driven Algorithm Design". In: *Beyond the Worst-Case Analysis of Algorithms*. Ed. by TimEditor Roughgarden. Cambridge University Press, 2021, pp. 626–645.

[11] Maria-Florina Balcan, Travis Dick, and Manuel Lang. "Learning to Link". In: *International Conference on Learning Representations*. 2020. URL: https://openreview.net/forum?id=S1eRbANtDB.

[12] Maria-Florina Balcan, Travis Dick, and Ellen Vitercik. "Dispersion for Data-Driven Algorithm Design, Online Learning, and Private Optimization". In: *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*. 2018, pp. 603–614. DOI: 10.1109/FOCS.2018.00064.

[13] Maria-Florina Balcan et al. "How much data is sufficient to learn high-performing algorithms? generalization guarantees for data-driven algorithm design". In: *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*. STOC 2021. Virtual, Italy: Association for Computing Machinery, 2021, pp. 919–932. ISBN: 9781450380539. DOI: 10.1145/3406325.3451036. URL: https://doi.org/10.1145/3406325.3451036.

[14] Maria-Florina Balcan et al. "Learning to Branch: Generalization Guarantees and Limits of Data-Independent Discretization". In: *J. ACM* 71.2 (Apr. 2024). ISSN: 0004-5411. DOI: 10.1145/3637840. URL: https://doi.org/10.1145/3637840.

[15] Maria-Florina Balcan et al. "Learning-Theoretic Foundations of Algorithm Configuration for Combinatorial Partitioning Problems". In: *Proceedings of the 2017 Conference on Learning Theory*. Ed. by Satyen Kale and Ohad Shamir. Vol. 65. Proceedings of Machine Learning Research. PMLR, July 2017, pp. 213–274. URL: https://proceedings.mlr.press/v65/balcan17a.html.

[16] Maria-Florina F Balcan, Travis Dick, and Colin White. "Data-Driven Clustering via Parameterized Lloyd's Families". In: *Advances in Neural Information Processing Systems*. Ed. by S. Bengio et al. Vol. 31. Curran Associates, Inc., 2018. URL: https://proceedings.neurips.cc/paper_files/paper/2018/file/128ac9c427302b7a64314fc4593430b2-Paper.pdf.

[17] Maria-Florina F Balcan, Anh Nguyen, and Dravyansh Sharma. "New Bounds for Hyperparameter Tuning of Regression Problems Across Instances". In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh et al. Vol. 36. Curran Associates, Inc., 2023, pp. 80066–80078. URL: https://proceedings.neurips.cc/paper_files/paper/2023/file/fd62b65606f0f0d2af2c01623a224258-Paper-Conference.pdf.

[18] Maria-Florina F Balcan, Tuomas Sandholm, and Ellen Vitercik. "Sample Complexity of Automated Mechanism Design". In: *Advances in Neural Information Processing Systems*. Ed. by D. Lee et al. Vol. 29. Curran Associates, Inc., 2016. URL: https://proceedings.neurips.cc/paper_files/paper/2016/file/c667d53acd899a97a85de0c201ba99be-Paper.pdf.

[19] Nina Balcan et al. "Provably tuning the ElasticNet across instances". In: *Advances in Neural Information Processing Systems*. Ed. by Alice H. Oh et al. 2022. URL: https://openreview.net/forum?id=ZMFQtvVJr40.

[20] Nina Balcan et al. “Structural Analysis of Branch-and-Cut and the Learnability of Gomory Mixed Integer Cuts”. In: *Advances in Neural Information Processing Systems*. Ed. by Alice H. Oh et al. 2022. URL: https://openreview.net/forum?id=e2gRdexoTZf.

[21] Evripidis Bampis, Bruno Escoffier, and Michalis Xefteris. “Canadian Traveller Problem with Predictions”. In: *Approximation and Online Algorithms*. Ed. by Parinya Chalermsook and Bundit Laekhanukit. Cham: Springer International Publishing, 2022, pp. 116–133. ISBN: 978-3-031-18367-6.

[22] Evripidis Bampis et al. “Scheduling with Untrusted Predictions”. In: *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*. Ed. by Lud De Raedt. Main Track. International Joint Conferences on Artificial Intelligence Organization, July 2022, pp. 4581–4587. DOI: 10.24963/ijcai.2022/636. URL: https://doi.org/10.24963/ijcai.2022/636.

[23] Peter Bartlett, Piotr Indyk, and Tal Wagner. “Generalization Bounds for Data-Driven Numerical Linear Algebra”. In: *Proceedings of Thirty Fifth Conference on Learning Theory*. Ed. by Po-Ling Loh and Maxim Raginsky. Vol. 178. Proceedings of Machine Learning Research. PMLR, Feb. 2022, pp. 2013–2040. URL: https://proceedings.mlr.press/v178/bartlett22a.html.

[24] J. M. Bates and C. W. J. Granger. “The Combination of Forecasts”. In: *OR* 20.4 (1969), pp. 451–468. ISSN: 14732858. URL: http://www.jstor.org/stable/3008764 (visited on 01/16/2025).

[25] Laszlo A. Belady. “A study of replacement algorithms for a virtual-storage computer”. In: *IBM Systems journal* 5.2 (1966), pp. 78–101.

[26] Avrim Blum, Chen Dan, and Saeed Seddighin. “Learning Complexity of Simulated Annealing”. In: *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics*. Ed. by Arindam Banerjee and Kenji Fukumizu. Vol. 130. Proceedings of Machine Learning Research. PMLR, 13–15 Apr 2021, pp. 1540–1548. URL: https://proceedings.mlr.press/v130/blum21a.html.

[27] Jan van den Brand et al. “On Dynamic Graph Algorithms with Predictions”. In: *Proceedings of the 2024 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 3534–3557. DOI: 10.1137/1.9781611977912.126. eprint: https://epubs.siam.org/doi/pdf/10.1137/1.9781611977912.126. URL: https://epubs.siam.org/doi/abs/10.1137/1.9781611977912.126.

[28] Vladimir Braverman et al. “Learning-Augmented Maximum Independent Set”. In: *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2024)*. Ed. by Amit Kumar and Noga Ron-Zewi. Vol. 317. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, 24:1–24:18. ISBN: 978-3-95977-348-5. DOI: 10.4230/LIPIcs.APPROX/RANDOM.2024.24. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.APPROX/RANDOM.2024.24.

[29] Yang Cai and Constantinos Daskalakis. “Learning Multi-item Auctions with (or without) Samples”. In: *CoRR* abs/1709.00228 (2017). arXiv: 1709.00228. URL: http://arxiv.org/abs/1709.00228.

[30] Ivan Carvalho and Ramon Lawrence. "LearnedSort as a learning-augmented SampleSort: Analysis and Parallelization". In: *Proceedings of the 35th International Conference on Scientific and Statistical Database Management*. SSDBM '23. Los Angeles, CA, USA: Association for Computing Machinery, 2023. ISBN: 9798400707469. DOI: 10.1145/3603719.3603731. URL: https://doi.org/10.1145/3603719.3603731.

[31] Shuchi Chawla and Dimitris Christou. "Online Time-Windows TSP with Predictions". In: *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2024)*. Ed. by Amit Kumar and Noga Ron-Zewi. Vol. 317. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, 2:1–2:21. ISBN: 978-3-95977-348-5. DOI: 10.4230/LIPIcs.APPROX/RANDOM.2024.2. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.APPROX/RANDOM.2024.2.

[32] Jakub Chłędowski et al. "Robust Learning-Augmented Caching: An Experimental Study". In: *Proceedings of the 38th International Conference on Machine Learning*. Ed. by Marina Meila and Tong Zhang. Vol. 139. Proceedings of Machine Learning Research. PMLR, 18–24 Jul 2021, pp. 1920–1930. URL: https://proceedings.mlr.press/v139/chledowski21a.html.

[33] Sami Davies et al. "Predictive Flows for Faster Ford-Fulkerson". In: *Proceedings of the 40th International Conference on Machine Learning*. Ed. by Andreas Krause et al. Vol. 202. Proceedings of Machine Learning Research. PMLR, 23–29 Jul 2023, pp. 7231–7248. URL: https://proceedings.mlr.press/v202/davies23b.html.

[34] Yuntian Deng et al. "Weighted gaussian process bandits for non-stationary environments". In: *International Conference on Artificial Intelligence and Statistics*. PMLR. 2022, pp. 6909–6932.

[35] Elbert Du, Franklyn Wang, and Michael Mitzenmacher. "Putting the "Learning" into Learning-Augmented Algorithms for Frequency Estimation". In: *Proceedings of the 38th International Conference on Machine Learning*. Ed. by Marina Meila and Tong Zhang. Vol. 139. Proceedings of Machine Learning Research. PMLR, 18–24 Jul 2021, pp. 2860–2869. URL: https://proceedings.mlr.press/v139/du21d.html.

[36] Paul Dütting et al. "Secretaries with advice". In: *Proceedings of the 22nd ACM Conference on Economics and Computation*. 2021, pp. 409–429.

[37] Jon C. Ergun et al. "Learning-Augmented k-means Clustering". In: *International Conference on Learning Representations*. 2022. URL: https://openreview.net/forum?id=X8cLTHexYyY.

[38] Paolo Ferragina, Fabrizio Lillo, and Giorgio Vinciguerra. "On the performance of learned data structures". In: *Theoretical Computer Science* 871 (2021), pp. 107–120. ISSN: 0304-3975. DOI: https://doi.org/10.1016/j.tcs.2021.04.015. URL: https://www.sciencedirect.com/science/article/pii/S0304397521002334.

[39] Amos Fiat et al. "Competitive paging algorithms". In: *Journal of Algorithms* 12.4 (1991), pp. 685–699.

[40] Amos Fiat et al. “Competitive paging algorithms”. In: *Journal of Algorithms* 12.4 (1991), pp. 685–699.

[41] Kaito Fujii and Yuichi Yoshida. “The Secretary Problem with Predictions”. In: *Mathematics of Operations Research* 49.2 (2024), pp. 1241–1262.

[42] Vasilis Gkatzelis et al. “Improved Price of Anarchy via Predictions”. In: *Proceedings of the 23rd ACM Conference on Economics and Computation*. EC ’22. Boulder, CO, USA: Association for Computing Machinery, 2022, pp. 529–557. ISBN: 9781450391504. DOI: 10.1145/3490486.3538296. URL: https://doi.org/10.1145/3490486.3538296.

[43] Sreenivas Gollapudi and Debmalya Panigrahi. “Online Algorithms for Rent-Or-Buy with Expert Advice”. In: *Proceedings of the 36th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri and Ruslan Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, Sept. 2019, pp. 2319–2327. URL: https://proceedings.mlr.press/v97/gollapudi19a.html.

[44] Yannai A. Gonczarowski and S. Matthew Weinberg. “The Sample Complexity of Up-to-ε Multi-dimensional Revenue Maximization”. In: *J. ACM* 68.3 (Mar. 2021). ISSN: 0004-5411. DOI: 10.1145/3439722. URL: https://doi.org/10.1145/3439722.

[45] Themistoklis Gouleakis, Konstantinos Lakis, and Golnoosh Shahkarami. “Learning-Augmented Algorithms for Online TSP on the Line”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 37.10 (June 2023), pp. 11989–11996. DOI: 10.1609/aaai.v37i10.26414. URL: https://ojs.aaai.org/index.php/AAAI/article/view/26414.

[46] Rishi Gupta and Tim Roughgarden. “A PAC approach to application-specific algorithm selection”. In: *Proceedings of the 2016 ACM Conference on Innovations in Theoretical Computer Science*. 2016, pp. 123–134.

[47] John L. Henning. “SPEC CPU2006 benchmark descriptions”. In: *SIGARCH Comput. Archit. News* 34.4 (Sept. 2006), pp. 1–17. ISSN: 0163-5964. DOI: 10.1145/1186736.1186737. URL: https://doi.org/10.1145/1186736.1186737.

[48] Chen-Yu Hsu et al. “Learning-Based Frequency Estimation Algorithms”. In: *International Conference on Learning Representations*. 2019. URL: https://openreview.net/forum?id=r1lohoCqY7.

[49] Sungjin Im et al. “Online Knapsack with Frequency Predictions”. In: *Advances in Neural Information Processing Systems*. Ed. by M. Ranzato et al. Vol. 34. Curran Associates, Inc., 2021, pp. 2733–2743. URL: https://proceedings.neurips.cc/paper_files/paper/2021/file/161c5c5ad51fcc884157890511b3c8b0-Paper.pdf.

[50] Sungjin Im et al. “Parsimonious Learning-Augmented Caching”. In: *Proceedings of the 39th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri et al. Vol. 162. Proceedings of Machine Learning Research. PMLR, 17–23 Jul 2022, pp. 9588–9601. URL: https://proceedings.mlr.press/v162/im22a.html.

[51] Zhihao Jiang, Debmalya Panigrahi, and Kevin Sun. “Online Algorithms for Weighted Paging with Predictions”. In: *ACM Trans. Algorithms* 18.4 (Oct. 2022).

ISSN: 1549-6325. DOI: 10.1145/3548774. URL: https://doi.org/10.1145/3548774.

[52] Zhihao Jiang, Debmalya Panigrahi, and Kevin Sun. "Online Algorithms for Weighted Paging with Predictions". In: *ACM Trans. Algorithms* 18.4 (Oct. 2022). ISSN: 1549-6325. DOI: 10.1145/3548774. URL: https://doi.org/10.1145/3548774.

[53] Murali Kodialam and T. V. Lakshman. "Prediction Augmented Segment Routing". In: *2021 IEEE 22nd International Conference on High Performance Switching and Routing (HPSR)*. 2021, pp. 1–6. DOI: 10.1109/HPSR52026.2021.9481858.

[54] Tim Kraska et al. "The Case for Learned Index Structures". In: *Proceedings of the 2018 International Conference on Management of Data*. SIGMOD '18. Houston, TX, USA: Association for Computing Machinery, 2018, pp. 489–504. ISBN: 9781450347037. DOI: 10.1145/3183713.3196909. URL: https://doi.org/10.1145/3183713.3196909.

[55] Silvio Lattanzi, Ola Svensson, and Sergei Vassilvitskii. "Speeding Up Bellman Ford via Minimum Violation Permutations". In: *Proceedings of the 40th International Conference on Machine Learning*. Ed. by Andreas Krause et al. Vol. 202. Proceedings of Machine Learning Research. PMLR, 23–29 Jul 2023, pp. 18584–18598. URL: https://proceedings.mlr.press/v202/lattanzi23a.html.

[56] Honghao Lin, Tian Luo, and David Woodruff. "Learning Augmented Binary Search Trees". In: *Proceedings of the 39th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri et al. Vol. 162. Proceedings of Machine Learning Research. PMLR, 17–23 Jul 2022, pp. 13431–13440. URL: https://proceedings.mlr.press/v162/lin22f.html.

[57] Evan Zheran Liu et al. "An imitation learning approach for cache replacement". In: *Proceedings of the 37th International Conference on Machine Learning*. ICML'20. JMLR.org, 2020.

[58] Pinyan Lu, Zongqi Wan, and Jialin Zhang. "Competitive Auctions with Imperfect Predictions". In: *Proceedings of the 25th ACM Conference on Economics and Computation*. EC '24. New Haven, CT, USA: Association for Computing Machinery, 2024, pp. 1155–1183. ISBN: 9798400707049. DOI: 10.1145/3670865.3673586. URL: https://doi.org/10.1145/3670865.3673586.

[59] Pinyan Lu et al. "Generalized Sorting with Predictions". In: *2021 Symposium on Simplicity in Algorithms (SOSA)*, pp. 111–117. DOI: 10.1137/1.9781611976496.13. eprint: https://epubs.siam.org/doi/pdf/10.1137/1.9781611976496.13. URL: https://epubs.siam.org/doi/abs/10.1137/1.9781611976496.13.

[60] Thodoris Lykouris and Sergei Vassilvitskii. "Competitive Caching with Machine Learned Advice". In: *J. ACM* 68.4 (July 2021). ISSN: 0004-5411. DOI: 10.1145/3447579. URL: https://doi.org/10.1145/3447579.

[61] Samuel McCauley et al. "Incremental Topological Ordering and Cycle Detection with Predictions". In: *Forty-first International Conference on Machine Learning*. 2024. URL: https://openreview.net/forum?id=wea7nsJdMc.

[62] Michael Mitzenmacher and Sergei Vassilvitskii. "Algorithms with predictions". In: *Communications of the ACM* 65.7 (2022), pp. 33–35.

[63] Mehryar Mohri and Andres Munoz Medina. "Learning Theory and Algorithms for revenue optimization in second price auctions with reserve". In: *Proceedings of the 31st International Conference on Machine Learning*. Ed. by Eric P. Xing and Tony Jebara. Vol. 32. Proceedings of Machine Learning Research 1. Bejing, China: PMLR, 22–24 Jun 2014, pp. 262–270. URL: https://proceedings.mlr.press/v32/mohri14.html.

[64] George V. Moustakides, Xujun Liu, and Olgica Milenkovic. "Optimal stopping methodology for the secretary problem with random queries". In: *Journal of Applied Probability* 61.2 (2024), pp. 578–602. DOI: 10.1017/jpr.2023.61.

[65] Thy Dinh Nguyen, Anamay Chaturvedi, and Huy Nguyen. "Improved Learning-augmented Algorithms for k-means and k-medians Clustering". In: *The Eleventh International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=dCSFiAl_VO3.

[66] Siddharth Prasad, Maria-Florina F Balcan, and Tuomas Sandholm. "Bicriteria Multidimensional Mechanism Design with Side Information". In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh et al. Vol. 36. Curran Associates, Inc., 2023, pp. 40832–40852. URL: https://proceedings.neurips.cc/paper_files/paper/2023/file/8039ca1e9860daab3a79e45d010d5398-Paper-Conference.pdf.

[67] Manish Purohit, Zoya Svitkina, and Ravi Kumar. "Improving Online Algorithms via ML Predictions". In: *Advances in Neural Information Processing Systems*. Ed. by S. Bengio et al. Vol. 31. Curran Associates, Inc., 2018. URL: https://proceedings.neurips.cc/paper_files/paper/2018/file/73a427badebe0e32caa2e1fc7530b7f3-Paper.pdf.

[68] Carl Edward Rasmussen. "Gaussian processes in machine learning". In: *Summer school on machine learning*. Springer, 2003, pp. 63–71.

[69] Dhruv Rohatgi. "Near-optimal bounds for online caching with machine learned advice". In: *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM. 2020, pp. 1834–1845.

[70] Tim Roughgarden. *Beyond the worst-case analysis of algorithms*. Cambridge University Press, 2021.

[71] Shinsaku Sakaue and Taihei Oki. "Generalization Bound and Learning Methods for Data-Driven Projections in Linear Programming". In: *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. 2024. URL: https://openreview.net/forum?id=jHh804fZ5l.

[72] Shinsaku Sakaue and Taihei Oki. "Improved Generalization Bound and Learning of Sparsity Patterns for Data-Driven Low-Rank Approximation". In: *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*. Ed. by Francisco Ruiz, Jennifer Dy, and Jan-Willem van de Meent. Vol. 206. Proceedings of Machine Learning Research. PMLR, 25–27 Apr 2023, pp. 1–10. URL: https://proceedings.mlr.press/v206/sakaue23a.html.

[73] Shinsaku Sakaue and Taihei Oki. "Sample Complexity of Learning Heuristic Functions for Greedy-Best-First and A* Search". In: *Advances in Neural Informa-*

tion Processing Systems. Ed. by Alice H. Oh et al. 2022. URL: https://openreview.net/forum?id=FurHLDnmC5v.

[74] Nicholas Schiefer et al. "Learned Interpolation for Better Streaming Quantile Approximation with Worst-Case Guarantees". In: *SIAM Conference on Applied and Computational Discrete Algorithms (ACDA23)*, pp. 87–97. DOI: 10.1137/1.9781611977714.8. eprint: https://epubs.siam.org/doi/pdf/10.1137/1.9781611977714.8. URL: https://epubs.siam.org/doi/abs/10.1137/1.9781611977714.8.

[75] Steve Seiden. "Unfair problems and randomized algorithms for metrical task systems". In: *Information and Computation* 148.2 (1999), pp. 219–240.

[76] Minjoon Seo et al. "Bidirectional Attention Flow for Machine Comprehension". In: *International Conference on Learning Representations*. 2017. URL: https://openreview.net/forum?id=HJ0UKP9ge.

[77] Dravyansh Sharma, Maria-Florina Balcan, and Travis Dick. "Learning piecewise Lipschitz functions in changing environments". In: *International Conference on Artificial Intelligence and Statistics*. PMLR. 2020, pp. 3567–3577.

[78] Yongho Shin et al. "Improved Learning-Augmented Algorithms for the Multi-Option Ski Rental Problem via Best-Possible Competitive Analysis". In: *Proceedings of the 40th International Conference on Machine Learning*. Ed. by Andreas Krause et al. Vol. 202. Proceedings of Machine Learning Research. PMLR, 23–29 Jul 2023, pp. 31539–31561. URL: https://proceedings.mlr.press/v202/shin23c.html.

[79] Daniel D Sleator and Robert E Tarjan. "Amortized efficiency of list update and paging rules". In: *Communications of the ACM* 28.2 (1985), pp. 202–208.

[80] Niranjan Srinivas et al. "Gaussian process optimization in the bandit setting: no regret and experimental design". In: *Proceedings of the 27th International Conference on International Conference on Machine Learning*. ICML'10. Haifa, Israel: Omnipress, 2010, pp. 1015–1022. ISBN: 9781605589077.

[81] *The 2nd Cache Replacement Championship*. [Online; accessed 14-January-2025]. 2017. URL: https://crc2.ece.tamu.edu/.

[82] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

[83] Vladimir Vovk, Alex Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, 2005. ISBN: 978-0387001524.

[84] Alexander Wei. "Better and Simpler Learning-Augmented Online Caching". In: *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2020)*. Ed. by Jarosław Byrka and Raghu Meka. Vol. 176. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, 60:1–60:17. ISBN: 978-3-95977-164-1. DOI: 10.4230/LIPIcs.APPROX/RANDOM.2020.60. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.APPROX/RANDOM.2020.60.

[85] Chenyang Xu and Pinyan Lu. "Mechanism Design with Predictions". In: *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-*

22. Ed. by Lud De Raedt. Main Track. International Joint Conferences on Artificial Intelligence Organization, July 2022, pp. 571–577. DOI: 10.24963/ijcai.2022/81. URL: https://doi.org/10.24963/ijcai.2022/81.

[86] Ali Zeynali, Shahin Kamali, and Mohammad Hajiesmaili. "Robust Learning-Augmented Dictionaries". In: *Forty-first International Conference on Machine Learning*. 2024. URL: https://openreview.net/forum?id=XyhgssAo5b.

[87] Ali Zeynali et al. "Data-driven Competitive Algorithms for Online Knapsack and Set Cover". In: *Proceedings of the AAAI Conference on Artificial Intelligence* 35.12 (May 2021), pp. 10833–10841. DOI: 10.1609/aaai.v35i12.17294. URL: https://ojs.aaai.org/index.php/AAAI/article/view/17294.

[88] Xingyu Zhou and Ness Shroff. "No-regret algorithms for time-varying bayesian optimization". In: *2021 55th Annual Conference on Information Sciences and Systems (CISS)*. IEEE. 2021, pp. 1–6.